Railway-Scheduling in Flatland
Modeling Accurate Speed In Curves

Victor Adamovski and Joel Böttcher

Institut für Informatik, Universität Potsdam

Abstract. Standard models for railway scheduling in Flatland often neglect the physical realities of velocity and track geometry. In this paper, we present an enhanced approach to implement a semi-realistic speed model using Clingo. We combine the modeling of more realistic curves with an approach to handle different speeds and linear acceleration/deceleration to simulate the speedlimits given by centripetal forces and friction in a curve. Therefore we created an iterative marker system for speed handling, a noise model calculated via OpenSimplex Noise in Python to recreate weather effects, and enforced the resulting speedlimits on a working environment. Our results demonstrate that via gapbridging to Python Clingo can be used to create a semi-realistic physical simulation in Flatland.

1 Introduction

The Flatland Challenge provides a robust framework for railway scheduling, yet it abstracts away the constraints of real-world environment to reduce computational complexity. While standard solvers treat transitions as uniform time steps, real-world railway operations are governed by strict physical laws of safety and motion. In this paper we propose an enhanced framework to integrate semi-realistic physical constraints into the Answer Set Programming (ASP) environment of Flatland. Our contribution has four contents. First we replace uniform movement with a dynamic velocity model that enforces linear acceleration and deceleration. Second we extend the discrete grid logic to support and approximate complex, multi-tiled curves beyond simple 90 degree transitions. Third we derive and enforce variable speedlimits based on centripetal force and friction. Finally we introduce a spatial and temporal noise model to simulate the impact of fluctuating weather conditions on rail friction. The remainder of this paper is organized as follows: Section 2 defines the core challenges of mapping continuous physics to discrete logic. Section 3 details our mathematical approach to curvature and velocity. Section 4 describes the hybrid Clingo-Python implementation and Section 5 concludes with a conclusion and outlook.

2 Problem Definition

In the Flatland environment, this work aimed to implement a semi-realistic speed model using ASP programming through Clingo. The objective was to improve

the realism of train movement within a discrete grid-based simulation. This task introduced two main challenges. First, we needed to define and compute train speeds in a discrete environment, where continuous motion and classical physical models cannot be directly applied. Second, we had to ensure the consistency of the movement by generating logical constraints that guarantee that the trains remain on valid tracks and behave correctly within the system.

3 Approach

In this section, we describe the approach used to implement a semi-realistic speed model in Flatland. Our approach consists of four main components:

- Speed modeling
- Curvature estimation
- Speed limit enforcement
- Environmental effects

In the following subsections, we will describe each component in detail.

3.1 Basic Train Movement

The fundamental requirement of the simulation is for trains to navigate from a starting point to a designated target station within a specific time window while avoiding collisions. In the Flatland environment, this involves managing "spawn" times and pathfinding through a discrete grid. Our initial asp implementation focused on ensuring that each train could calculate a valid sequence of actions to reach its destination without violating global temporal constraints. To ensure that each of the trains gets a unique spawning time, we gave every train that started at the same station a random time where no other train spawns after their initial departure time. The path finding problem was solved by creating a set of actions the trains could choose from depending on the track beneath. After choosing a move, the code validates if it is a possible move and saves the new position of the train. To ensure there were no crashes we made all positions invalid where two trains are at the same position or swap there position during a timestep. And with that all trains should traverse the network successfully.

3.2 Speed Modeling

To transition from a default uniform movement to a more realistic representation of speed, we implemented two distinct computational models. The primary goal was to allow trains to vary their travel time between tiles, effectively simulating different speeds within a discrete grid. For easier use, we implemented the inverse speed, which allowed the use of time steps needed for a transition from one grid cell to the other as a representation for the current speed.

First Approach (Recursive) Our first approach was to define the speed in a recursive logical derivation of all the chosen actions. In the model, we had a base case of movement defined as one which every action needs. Recursively, we added a wait action after the movement to increase the speed of this movement by one. And to have a maximum we choose the speed of four as the cap with any remaining gaps filled in with five for showing no movement. Although functional, the system had downsides. The approach proved computationally inefficient in comparison to our second approach due to choosing the independent movements before hand and recursively calculating the inverse speed in advance. And secondly, the movement happened in the first space of a recursive build speed block before the train waited, which was not equivalent to our understanding of speed.

Second Approach (Iterative) To truly ensure the speed we wanted, we needed a new approach. Instead of recursively building up the speed, we calculate our speed based on the speed used by the action before. As starting point for an action we used a marker system which marked where the last action has ended. For each marker, the system chooses a speed and adds a block of movement ticks associated with the chosen speed. So for a speed of four the block consists first of three waits and then of the chosen action. This simple but critical structural change ensures that the model aligns with real-world physical systems: the train only arrives at the new track segment after the total required traversal time has elapsed. The system itself is also faster than the first approach and therefore makes the adoption of changes in the environment more robust.

3.3 Acceleration and Deceleration

With the calculated speed, we can apply realistic limitations to the speed of the trains. The simplest is a limited acceleration and deceleration. Enforcing smooth speed changes is done by comparing speeds in consecutive time steps. If it differs by more than 1, the solution is discarded, as the train accelerates or decelerates faster than it should be able to.

3.4 Curvature Estimation

Curves are another influence on a train's speed. If a train goes too fast on a curve, it will derail. On a perfect circle, the allowed speed can be calculated from the radius of the circle. The simplest approach is to view tiles individually, with straight rails imposing no speed limit and curved rails having a fixed radius, imposing a constant speed limit. While this yields accurate results for the Flatland environment, it creates limitations for modelling realistic environments. Our solution is to view the discrete paths presented in the Flatland environment as smooth curves, which comes with its own challenges.

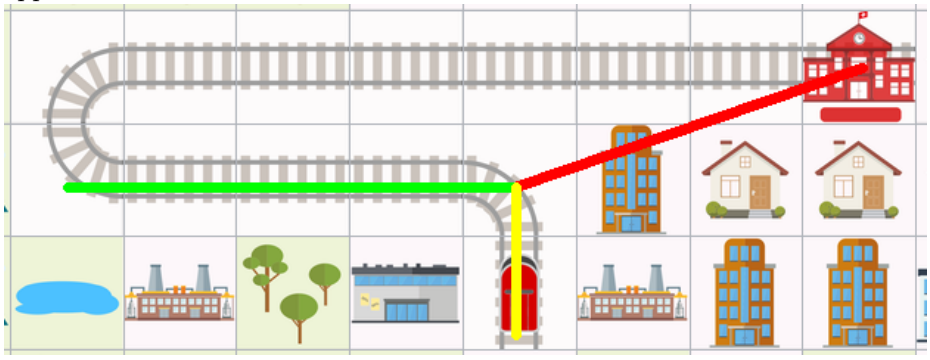
Curves A curve is defined as a function mapping a parameter t to a position in (in this case 2-dimensional) space. A path between two tiles in the environment can be represented as a curve. To smoothen the curve, we only consider connections between curved rails of the same type. Connections are determined for each curved rail similarly to the floodfill algorithm. If a tile contains multiple curved parts, connections to those parts are determined separately. The connection from one curved rail part to the next, expressed as a vector, will be referred to as its extension. A part of a track might be characterized by multiple curves. For example, a large (multi-tile) turn (viewed such that it turns right) consists of alternating right and left turns. This approach creates a curve of outer (right) turns and a curve of inner (left) turns. Speed limits are applied by both, effectively only considering the stricter (inner/left) curve.

The extension of a rail part p at (x, y) is determined as follows:
First, find all extparts (extension parts) of p:
- $(0, 0)$ is an extpart of p
- $(dx + ddx, dy + ddy)$ is an extpart of p, if the following conditions are met:

(ddx, ddy) is a valid step from (dx, dy) A valid step is a vector of magnitude one, representing movement by one tile. Additionally, a rail must connect the tiles connected by the step.

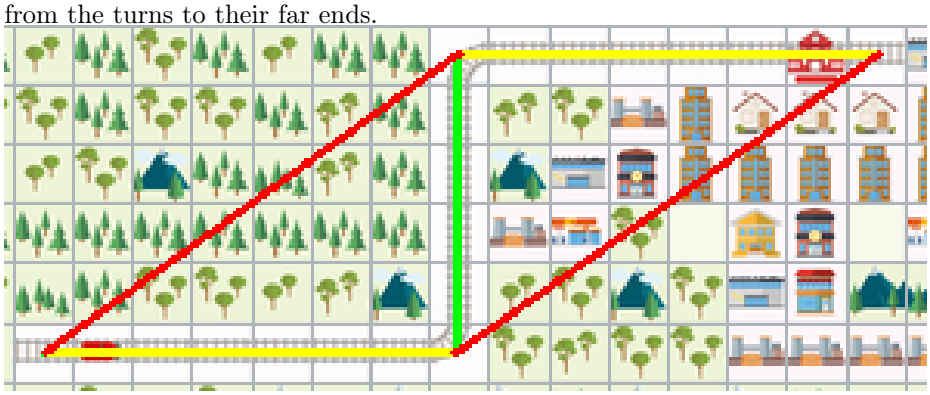
Tile $(x + dx, y + dy)$ doesn't contain p Unless on the starting tile, finding a copy of the original curve as part of the previously visited tile terminates the search.

No going back The search starts in a certain direction (dx_0, dy_0) given by the shape of p. The rail might go perpendicularly for 1 step but make a sharp turn back instead of continuing the same direction with a p-shaped rail. The extension should stop at the sharp turn, as following it would cut it out of the approximation.



(ddx, ddy) isn't perpendicular to the start twice Changing direction is expected when following a curve. Deviating from the original direction once

indicates a minimal turn, soon followed by a curved rail similar to p leading back. However, if the track goes off perpendicularly to the curve's start for at least 2 steps, we should not follow that section. For example, if we have a left turn followed by a right turn with long, straight paths in between, we should treat the turns as the 90-degree turns they are instead of placing smooth diagonals from the turns to their far ends.



with perpendiculars (inaccurate)
without perpendiculars
The extensions of p are its maximum extparts.

Curvature The curvature κ of a rectified curve g in a point t is defined as follows:

$$\kappa = ||g''(t)|| = \left|\left|\frac{d^2g}{dt^2}\right|\right|$$

While our new curve model is not bound to the 90-degree turns of the grid, it is still straight in most places and singular in others, requiring an approximation for the derivative. We are only measuring curvature on the curved rails as the straight rails represent a part of the real world track that lies between its limiting curved rails in curvature. The first derivative g' is determined by the extensions of the curve. In points before and after t, the derivative's direction matches the corresponding extension. A curved rail has one extension for each end, labeled ext_0 ("before" t) and ext_1 ("after" t). To get the rectified curve required to apply the definition of curvature, the extensions are normalized, resulting in $\overline{ext_0}$ and $\overline{ext_1}$. In t, the first derivative isn't defined or computed. Instead, the second derivative is directly approximated using a discrete version of the symmetrical derivative:

$$f'(x) = lim_{h\to 0}\frac{f(x+h) - f(x-h)}{2h}$$

Applied to the curve, we get

$$g''(t) = lim_{h\to 0}\frac{g'(t+h) - g'(t-h)}{2h}$$

As established, g' is defined through exts: $g'(t+h) = \overline{ext_1}$ and $g'(t-h) = -\overline{ext_0}$. The negation is required because g' follows the curve's path from beginning to

end, pointing into the curved rail at $t-h$, while ext_0 points away from the curved rail. Plugging the values in yields

$$g''(t) = lim_{h\to 0}\frac{\overline{ext_1} + \overline{ext_0}}{2h}$$

The final variable that needs to be addressed is h. The limit fails in this scenario as $||ext_1 + ext_0||$ is a positive real number, meaning the limit would evaluate to a vector with at least one infinite element. Instead, we need to get the value we used for dt, represented by h. The ext vectors have a length of 1 from rail to the other endpoint, making that our dt. From this, we derive the final compact equation for curvature:

$$\kappa = ||g''(t)|| = \left|\left|\frac{\overline{ext_1} + \overline{ext_0}}{2}\right|\right|$$

We can further specify κ by expressing it in terms of values accessible in clingo:

$$ext_i = (x_i, y_i)$$

$$\overline{ext_i} = \frac{ext_i}{||ext_i||} = \left(\frac{x_i}{\sqrt{x_i^2 + y_i^2}}, \frac{y_i}{\sqrt{x_i^2 + y_i^2}}\right)$$

$$||\overline{ext_0} + \overline{ext_1}|| = \left|\left|\frac{ext_0}{||ext_0||} + \frac{ext_1}{||ext_1||}\right|\right|$$

$$2\kappa = \left|\left|\begin{pmatrix}\frac{x_0}{||ext_0||} + \frac{x_1}{||ext_1||}\\ \frac{y_0}{||ext_0||} + \frac{y_1}{||ext_1||}\end{pmatrix}\right|\right|$$

$$2\kappa = \sqrt{\left(\frac{x_0}{||ext_0||} + \frac{x_1}{||ext_1||}\right)^2 + \left(\frac{y_0}{||ext_0||} + \frac{y_1}{||ext_1||}\right)^2}$$

Due to κ always being non-negative, we can remove the square root by squaring both sides:

$$4\kappa^2 = \left(\frac{x_0}{||ext_0||} + \frac{x_1}{||ext_1||}\right)^2 + \left(\frac{y_0}{||ext_0||} + \frac{y_1}{||ext_1||}\right)^2$$

We can now resolve the squares by applying the binomial formula $(a+b)^2 = a^2 + b^2 + 2ab$ and grouping terms with the same denominator:

$$4\kappa^2 = \frac{x_0^2 + y_0^2}{||ext_0||^2} + \frac{x_1^2 + y_1^2}{||ext_1||^2} + \frac{2(x_0y_0 + x_1y_1)}{||ext_0|| \cdot ||ext_1||}$$

The terms created by the a^2 and b^2 parts are equal to 1 by definition of ext_0 and ext_1, simplifying the equation to

$$4\kappa^2 = 2 + \frac{2(x_0y_0 + x_1y_1)}{||ext_0|| \cdot ||ext_1||}$$

Expanding the *ext*s in the denominator yields

$$4\kappa^2 = 2 + \frac{2(x_0y_0 + x_1y_1)}{\sqrt{x_0^2 + y_0^2} \cdot \sqrt{x_1^2 + y_1^2}}$$

The part consisting of only x_i and y_i will be referred to as C. Note that while $4\kappa^2$ is always non-negative, the same cannot be said about C, which can be as low as -1.

$$4\kappa^2 = 2 + 2C$$
$$\kappa^2 = \frac{1}{2} + \frac{1}{2}C$$

3.5 Speed limit on Curves

The curvature of a curve has another useful interpretation: it is the inverse of the radius of the circle approximating the curve. We can use the circle obtained this way to compute a speed limit for the curve. Movement on a circle requires a constant centripetal acceleration caused by centripetal force. Without it, the train would continue travelling in a straight line and derail from the curve. In our scenario, this force is provided by friction with the rails, so we set the forces equal:

$$F_f = F_c$$
$$\mu mg = \frac{mv^2}{r}$$

Rearranging reveals the following formula:

$$v = \sqrt{r\mu g}$$

The speed obtained this way is the upper limit; the train's speed v can be lower:

$$v \leq \sqrt{r\mu g}$$

We also need to account for Flatland's usage of inverse speed by inverting the resulting inequality:

$$v^{-1} \geq \sqrt{r^{-1}\mu^{-1}g^{-1}}$$

Due to clingo only operating on integers, we need to represent μ and g as rational numbers constructed from integers:

$$\mu = \frac{\hat{\mu}}{\check{\mu}} \; and \; g = \frac{\hat{g}}{\check{g}}$$

We can now plug the other inverse values into the equation: inverse radius is curvature by definition and for the other constants, we invert the fractions:

$$v^{-1} \geq \sqrt{\frac{\kappa\check{\mu}\check{g}}{\hat{\mu}\hat{g}}}$$

Another limitation imposed by clingo's integer operations is that square roots and divisions lead to rounding errors. These operations can be circumvented by rearranging the formula to use squares and multiplication instead. As both sides are always non-negative, we can square the inequality without creating extraneous solutions. We square it twice to apply the definition of κ without adding roots:

$$v^{-4} \geq \left(\frac{\kappa\check{\mu}\check{g}}{\hat{\mu}\hat{g}}\right)^2$$

$$v^{-4} \geq \left(\frac{1}{2} + \frac{1}{2}C\right) \cdot \left(\frac{\check{\mu}\check{g}}{\hat{\mu}\hat{g}}\right)^2$$

Next, we remove divisions by multiplying by all positive constants in denominators:

$$2(\hat{\mu}\hat{g})^2 v^{-4} \geq (1 + C) \cdot (\check{\mu}\check{g})^2$$

$$2(\hat{\mu}\hat{g})^2 v^{-4} \geq (\check{\mu}\check{g})^2 + C\,(\check{\mu}\check{g})^2$$

$$2(\hat{\mu}\hat{g})^2 v^{-4} - (\check{\mu}\check{g})^2 \geq C\,(\check{\mu}\check{g})^2$$

For further reference, the left-hand side will be named $l(v)$:

$$l(v) \geq C\,(\check{\mu}\check{g})^2$$

Applying the definition of C reintroduces further divisions and square roots. While the divisions are easily resolvable by multiplying, a simple squaring will not resolve the square roots, as both sides can now be negative. Additional conditions will be required.

$C(\check{\mu}\check{g})^2$ represents a potentially negative threshold on $l(v)$. If the threshold is positive, we need to remove the higher values created by squaring $l(v)$. We can see that the derivative of $l(v)^2$ is negative everywhere where v^{-4} is positive and $l(v)$ is negative. We can add the derivative being positive as a condition:

$$\frac{d}{dv^{-1}} l(v)^2 > 0$$

$$\frac{d}{dv^{-1}} \left(2(\hat{\mu}\hat{g})^2 v^{-4} - (\check{\mu}\check{g})^2\right)^2 > 0$$

$$2\left(2(\hat{\mu}\hat{g})^2 v^{-4} - (\check{\mu}\check{g})^2\right) \cdot \frac{d}{dv^{-1}} \left(2(\hat{\mu}\hat{g})^2 v^{-4} - (\check{\mu}\check{g})^2\right) > 0$$

$$2\left(2(\hat{\mu}\hat{g})^2 v^{-4} - (\check{\mu}\check{g})^2\right) \cdot 4\left(2(\hat{\mu}\hat{g})^2 v^{-3}\right) > 0$$

With this condition added, we can replace $l(v) \geq C\,(\check{\mu}\check{g})^2$ with the squared version:

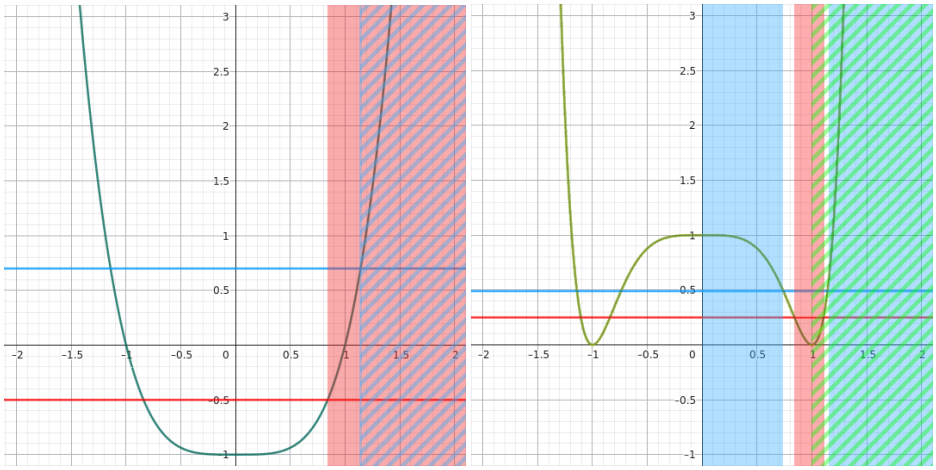
$$l(v)^2 \geq C^2\,(\check{\mu}\check{g})^4$$

. If the threshold is negative, all values where the left-hand side was positive fulfill the equation. Some values, where $l(v) < 0$ still need to be captured as

$\frac{d}{dv^{-1}} l(v)^2$ is negative there. We can apply an inverted version of the inequality as sufficient: $l(v)^2 \leq C^2 \left(\check{\mu}\check{g}\right)^4$ In summary, these are the conditions for a speed to be valid in a clingo-usable form:

$$C \left(\check{\mu}\check{g}\right)^2 \geq 0 \wedge \left(\frac{d}{dv^{-1}} l(v)^2 > 0 \wedge l(v)^2 \geq C^2 \left(\check{\mu}\check{g}\right)^4\right) \vee$$

$$C \left(\check{\mu}\check{g}\right)^2 < 0 \wedge \left(\frac{d}{dv^{-1}} l(v)^2 > 0 \vee l(v)^2 \leq C^2 \left(\check{\mu}\check{g}\right)^4\right)$$



The red and blue lines show possible thresholds $C \left(\check{\mu}\check{g}\right)^2$ for $l(v)$. The left graph shows $l(v)$; the colored areas show valid inverse speeds for their respective thresholds. The right graph shows $l(v)^2$ with squared thresholds. We can observe that the conditions hold: the values for inverse speed over the red (negative) threshold are in the red area ($l(v)^2 \leq C^2(\check{\mu}\check{g})^4$) **or** the green area ($\frac{d}{dv^{-1}} l(v)^2$), while the values over the blue threshold are in the blue ($l(v)^2 \geq C^2(\check{\mu}\check{g})^4$) **and** green areas.

The conditions not containing C have already been written in a clingo-usable form. However, C still contains division and square roots. When computing $C(\check{\mu}\check{g})^2$, the denominator and positive constants can be omitted as they do not change the sign of the expression, reducing the comparisons to $(x_0 y_0 + x_1 y_1) \geq 0$ and $(x_0 y_0 + x_1 y_1) < 0$ respectively. Rearranging the final term in each option is now possible due to C^2 not having the square roots C has. Thus, it can be expressed as follows:

$$l(v)^2 \geq C^2 \left(\check{\mu}\check{g}\right)^4$$

$$\left(2(\hat{\mu}\hat{g})^2 v^{-4} - \left(\check{\mu}\check{g}\right)^2\right)^2 \geq C^2 \left(\check{\mu}\check{g}\right)^4$$

$$\left(2(\hat{\mu}\hat{g})^2 v^{-4} - \left(\check{\mu}\check{g}\right)^2\right)^2 \geq \left(\frac{(x_0 y_0 + x_1 y_1)}{\sqrt{x_0^2 + y_0^2} \cdot \sqrt{x_1^2 + y_1^2}}\right)^2 \left(\check{\mu}\check{g}\right)^4$$

$$\left(2(\hat{\mu}\hat{g})^2 v^{-4} - \left(\check{\mu}\check{g}\right)^2\right)^2 \geq \frac{(x_0 y_0 + x_1 y_1)^2}{(x_0^2 + y_0^2)(x_1^2 + y_1^2)} \left(\check{\mu}\check{g}\right)^4$$

Multiplication with the non-negative denominator creates an inequality only relying on exact integer operations:

$$(x_0^2 + y_0^2)(x_1^2 + y_1^2) \left(2(\hat{\mu}\hat{g})^2 v^{-4} - (\check{\mu}\check{g})^2\right)^2 \geq (x_0 y_0 + x_1 y_1)^2 (\check{\mu}\check{g})^4$$

Using this formula in clingo has one final issue: clingo uses 32-bit integers, which can store numbers up to 2147483647, for all integer operations. Our new formula can exceed this value under certain conditions:

The first two terms on the left-hand side scale quadratically with extension length. Due to the normalization of extensions being built into other parts of the formula, long stretches of track will apply their squared length as a factor. This can be mitigated by stopping the extension search after a certain number of tiles have been traveled, at the cost of curve accuracy.

The third term scales with v^{-8}. Using our speed rules, inverse speed can range from 1 to 4, creating a factor of up to 256. Larger inverse speeds would quickly increase it even further.

The third term scales with μ^4 and g^4 for both numerators and denominators. Using values with high resolution results in a large denominator and an even larger factor.

The right-hand side has similar dangers to points 1 and 3, but it is less likely to overflow due to the lack of inverse speed.

3.6 Weather/Noise Model

After we have a working concept with a more or less static value for μ for a speed limit on the curves, we decided to go a step further and implement some kind of weather influence. In our real world, the weather is never steady and has a great impact on the friction of the tracks. Although moving steel wheels on steel tracks usually have a friction around 0.2, under icy conditions it can realistically drop to 0.03. To simulate a somehow realistic weather change in time and not drastically depending on the place in the environment, we choose to simulate it through a noise function. The generated noise itself ensures there that the conditions are random but smoothly along the whole grid, mimicking how the real weather front moves. The now generated value for μ is fed into the before stated speed limit calculation and therefore has an impact on the maximum allowed speed of a track, just as icy or dry conditions do in our real world.

4 Implementation

To translate our theoretical concepts into a functional simulation, we developed a multi-stage framework consisting of an Answer Set Programming (ASP) core and a Python-based environmental generator. This section details the technical realization of our physical models.

4.1 Redefining constants for Flatland

In the grid of Flatland some of our standard physical constants like the gravitational acceleration g do not exist anymore. To maintain our approach of realism we established a mapping between the grid tiles and real-world units. One tile we defined as 8 meters based on standardized rail dimensions. Secondly we set our maximum train speed at 170 km/h or 48 m/s which is around the upper limit of a regional train. Based on this maximum and the optimal inverse speed of one we derived that one simulation tick represents 1/6 of a second. Consequently, we calculated the Flatland constant for g as follows:

$$g \approx 9.81 \cdot \frac{0.125 tiles}{6^2 tick^2} \approx \frac{2}{59} \frac{tile}{tick^2}$$

In the ASP Code itself g is represented as rational number using numerator Gn and denumerator Gd to avoid floating point errors.

4.2 Speed Predicates and Temporal Consistency

The iterative speed model is implemented via a marker system that ensures trains spend time on a tile relative to their velocity like stated in 3.2. The predicate `speedpart(ID, T, N)` manages these discrete blocks. To ensure the train arrives at a new tile after the traversal time has elapsed, we placed "wait" actions at the beginning of each movement block. This structure is enforced by the `speed2 (ID, T, N)` predicate, which acts as the final velocity state. Furthermore, linear acceleration is strictly enforced through integrity constraints that prune any solution where the speed difference between consecutive ticks exceeds one of the following:

Listing 1.1. Speed integrity constraint

```
1 :- speed(ID,T,N1), speed2(ID,t+1,N2), N1=1..5, N1+1<N2, time(
      T+1).
2 :- speed(ID,T,N1), speed2(ID,t+1,N2), N1=1..5, N1-1>N2, time(
      T+1).
```

4.3 Integer-Based Curvature Logic

Implementing the speed limit $v^{-1} \geq \sqrt{r^{-1}\mu^{-1}g^{-1}}$ presented a significant challenge as Clingo operates on Integers and completely without floats. As derived in 3.5, we transformed this into a high-degree polynomial inequality to eliminate square roots and divisions. In our old `curvature.lp`, this is implemented as stated in the last derived formula. To prevent Integer overflows caused by terms scaling with v^{-8} or g^4, we used a fixed-point representation for friction (μ) and gravity (g). The core of the enforcement is the following integrity constraint:

Listing 1.2. complete in Clingo calculated allowed integrity constraint

```
% v: inverse speed, n: numerator, d: denominator
allowed(X, Y, Part, V) :- ext(Part, X, Y, Dir0, DX0, DY0),
    ext(Part, X, Y, Dir1, DX1, DY1), curve(Part, Dir0, Dir1),
    V=1..40, mu(X, Y, 1, MUn, MUd), g(Gn, Gd), % n: numerator
        , d: denominator
    (DX0**2+DY0**2) * (DX1**2+DY1**2) * (4*V**4*MUn*Gn - 2*
        MUd*Gd) ** 2 >= MUd**2 * Gd**2 * 4 * (DX0*DX1 + DY0*
        DY1) ** 2,
    (V**4 * MUn * Gn - 2 * MUd * Gd) > 0, % derivative;
        ensure V is on rising part
    DX0*DX1 + DY0+DY1 > 0. % fix square
```

While this part shows that it is doable to create a complex formula of floating numbers via Integers in Clingo, it felt completely impractical. So we revoked this decision later and implemented (upon identifying the embedded Python scripting abilities in Clingo) a Python function into the Clingo script to handle it way more effective. With that change our current code looks like the following.

Listing 1.3. Allowed integrity constraint with limit script in Clingo

```
% v: inverse speed, n: numerator, d: denominator
allowed(X, Y, T, Part, V) :- ext(Part, X, Y, Dir0, DX0, DY0),
     ext(Part, X, Y, Dir1, DX1, DY1), curve(Part, Dir0, Dir1)
    ,
    V=1..40, mu(X, Y, T, MUn, MUd), g(Gn, Gd),
    V > @limit(DX0, DX1, DY0, DY1, Gn, Gd, MUn, MUd).
```

Limit is defined in the following block via a Python call in Clingo itself, which mitigated the precision issues inherent in integer-only calculations.

Listing 1.4. Pythonscript to calculate the limit in Clingo

```
%n: numerator, d: denominator

#script (python)
import clingo

sqrt = lambda x: x ** 0.5  #functional lambda approximation

def limit(X0, X1, Y0, Y1, Gn, Gd, MUn, MUd):
    try:
        res = sqrt(MUd.number/MUn.number * Gd.number/Gn.
            number) * sqrt(0.5 + 0.5 * (X0.number*X1.number +
             Y0.number*Y1.number) / (sqrt(X0.number**2 + Y0.
            number**2)*sqrt(X1.number**2 + Y1.number**2)))
        print(res)
        return clingo.Number(int(res))
    except ZeroDivisionError:
        return clingo.Number(0)
#end.
```

The resulting logic allows the solver to verify complex curvature and their speed limits.

4.4 Environmental Noise Integration

To simulate realistic, dynamic weather conditions, we implemented an external Python module `NoiseCalculation.py`. Static friction values would not adequately challenge the speed-limit logic. Therefore we utilized the OpenSimplex noise library to generate a three-dimensional field mapping x, y and time t. Unlike pure random number generation, Simplex noise provides spatial and temporal coherence, mimicking how actual weather fronts move across a landscape. As shown in the following snippet, the continuous noise values are normalized and scaled to match physical properties of rail friction, where μ typically ranges from 0.2 (optimal dry) to 0.03 (severe icing). These values are then discretized into the integer numerators (MUn) and denominators (MUd) required by our ASP curvature logic:

Listing 1.5. Noisefunction in Python

```
#Normalize noise to [0, 1], scale by 10, and shift to avoid
    zero friction
#nRaum and tTime are changable variables to change temporal
    and spatial coherence

    muMatrix=opensimplex.noise3array((np.array(range(x))/
        nRaum),(np.array(range(y))/nRaum),(np.array(range(
        T_max))/tTime))
    muMatrix=(((muMatrix+1)*10//1)+1)
    MUd=1 #Constant denominator for ASP rational arithmetic
```

After the denominators and numerators were calculated, the program had written it in the right facts format (`mu(X, Y, T, MUn, MUd)`) back in a new file. By separating the noise generation from the logic solver, we allow for complex environmental pre-calculations without increasing the grounding time of the core physics framework.

5 Conclusion and Outlook

This paper demonstrates that the combination of Clingo and Python within the Flatland framework provides a robust foundation for simulation realistic railway environments. By implementing a hybrid architecture we successfully integrated complex physical constraints, such as centripetal force and dynamic friction, into a discrete logic solver. However, the current model has limitations that provide opportunities for further research. The primary constraint is the lack of specialized environment generator capable of placing the multi-tile curves our model supports. Currently these must be placed by hand. Furthermore, while the OpenSimplex noise model provides spatial coherence, the weather generation remains semi-stochastic and does not yet account for advanced weather patterns or seasonal trends. To build upon this research, our first objective is to develop a visual overlay for the Flatland environment. This would provide feedback on active speed limits and fluctuating weather, making the solver's decision more transparent. Secondly, we intend to develop a custom environment generator specifically designed to utilize our approach to complex track geometries. Finally we aim to introduce interactive mechanics between the trains and the environment. For instance could a thermodynamic model where the generated heat by passing trains have an influence on the weather effects like melting ice potentially have an impact in improving friction for the following trains. This would also mean that we apply all of our speed limits in real time on the grid and not like now just the slowest speedlimit which is possible on the track.